

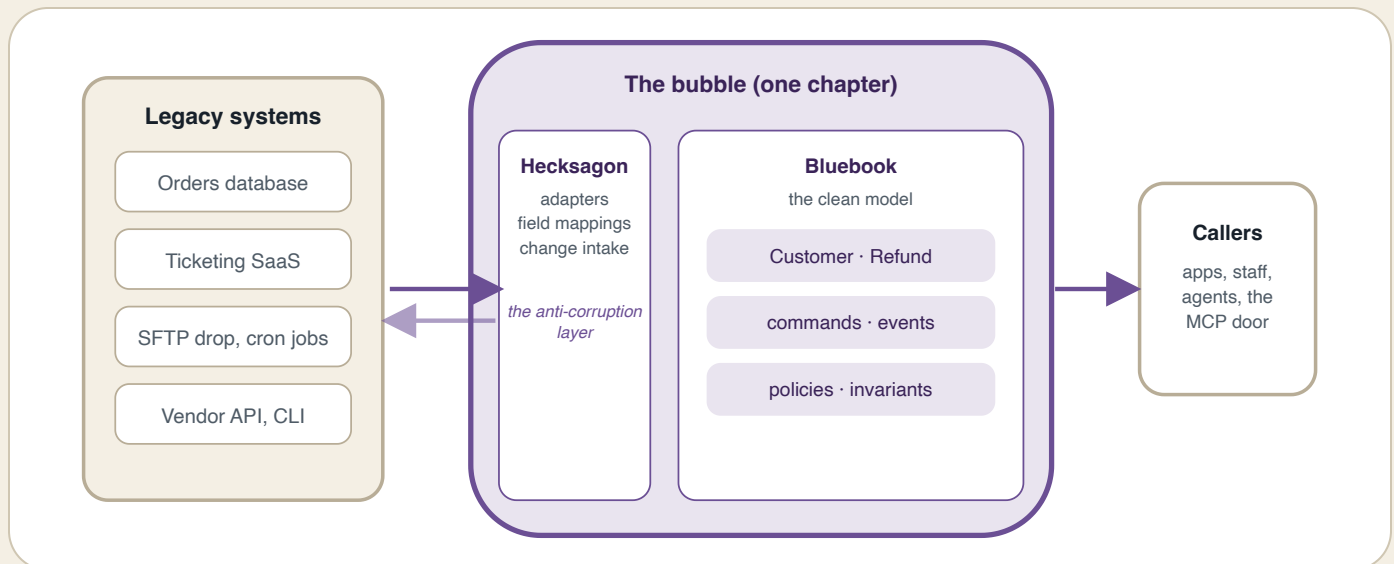


Hecks Bubbles

A clean model for one slice of the business, wrapped around the systems a company already runs, and grown until the old ones can go.

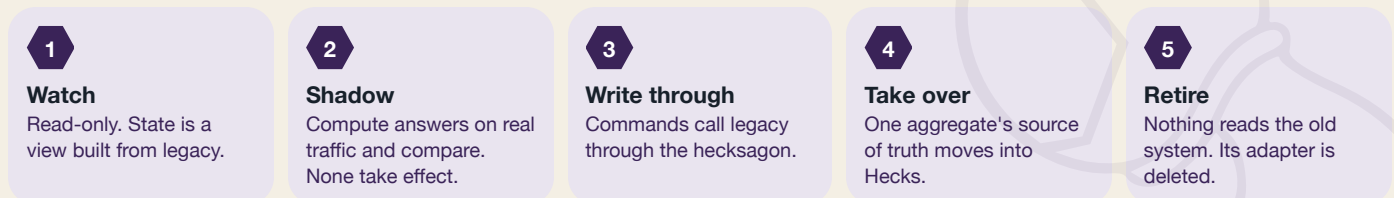
A bubble is a chapter wrapped around legacy

Evans called it a bubble context: a small, clean model protected from a legacy system by an anti-corruption layer. In Hecks it needs no new keyword. It is a chapter loaded into a host, whose hexsagon binds to the legacy systems.



PART	HOLDS	SUPPLIED BY
Bluebook	The clean model and its ports. Says nothing about any one company.	Shared, versioned
Hecksagon	Which adapter fulfils each port, plus the mappings from legacy shapes. This is the anti-corruption layer.	The company
World	Per-environment settings: URLs, credential references, limits, the read-only switch.	The company, per environment

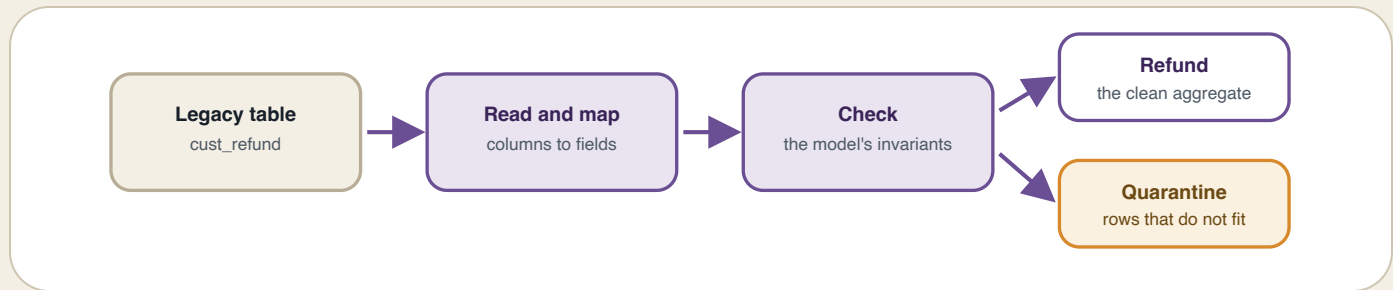
Five steps from watching to retiring



WORKED EXAMPLE

Consuming legacy data DESIGN SKETCH, NOT BUILT

A legacy orders system keeps refunds in one table with cryptic columns and one-letter status codes. The bubble reads the rows, maps them into a clean **Refund**, and sets aside any row that does not fit. Nothing is written back.



1. What legacy holds

RFND_ID	CUST_NO	AMT_CENTS	STAT_CD
10041	C-882	2500	P
10042	C-910	1800	A
10043	C-882	(null)	Z

2. The clean model (bluebook)

```
aggregate "Refund" do
  identified_by :refund_id
  attribute :refund_id, String
  attribute :customer, String
  attribute :amount, Money
  attribute :status, RefundStatus
  # Money (cents >= 0) and RefundStatus
  # (pending, approved, rejected) omitted
end
```

3. The mapping (hecksagon)

```
cust_refund -> Refund
RFND_ID     -> refund_id
CUST_NO     -> customer
AMT_CENTS   -> amount.cents
STAT_CD     -> status
P: pending  A: approved
X: rejected
```

4. What the bubble ends up with

REFUND	AMOUNT	STATUS
10041	2500	pending
10042	1800	approved

QUARANTINED	WHY
10043	no amount; status Z maps to nothing

Why this order. The mapping and the model's invariants decide what counts as a refund, so dirty rows never become aggregates. The quarantine keeps the reason, so a person can fix the source or extend the mapping and a re-read picks the row up. The table is only read, so this is safe against production on day one.

The plumbing

- **Thin wrappers.** Each prewritten adapter calls a shared Http, Cli or protocol client and adds only its service's mapping, as data read by both the Ruby and Rust runtimes.
- **Catalog.** Payments, email, chat, tickets, CRM, ERP, files, identity, cloud CLIs, databases (SQL, Supabase), queues, webhooks, SOAP.
- **Checked.** The model check refuses an unbound port or a missing setting.

Next, and still open

- **Build order.** Port and settings checks, then the transport clients, then Stripe, Slack, gh, SQL and webhooks, then importer, shadow mode and the drift report.
- **Open.** Mappings in the hecksagon or the world; one hecksagon per chapter or per host; how read-only is enforced; and whether a journal-less table is read as a projection, since persistence is append-before-project.